

UNIVERSITÀ DEGLI STUDI DI  
MILANO-BICOCCA

ADVANCED MACHINE LEARNING  
FINAL PROJECT

---

# Images Rotation Classification

---

*Author:*

Pizzo Davide - 869254- d.pizzo@campus.unimib.it

January 2026



## Abstract

Il progetto svolto affronta il task della classificazione dell'orientamento delle immagini, che può essere  $0^\circ$ ,  $90^\circ$ ,  $180^\circ$  e  $270^\circ$ .

L'obiettivo è quindi, usare diverse strategie di apprendimento e valutarne le prestazioni. Il lavoro si divide principalmente in 2 approcci, che saranno spiegati successivamente nella relazione.

## 1 Introduzione

La classificazione automatica dell'orientamento delle immagini rappresenta un problema interessante in diversi ambiti della computer vision, come per esempio l'organizzazione automatica di librerie fotografiche o l'elaborazione di dataset per sistemi di riconoscimento visivo. Questo progetto affronta un task di classificazione multi-classe con l'obiettivo di determinare se un'immagine è visualizzata correttamente o se risulta essere ruotata di  $90^\circ$ ,  $180^\circ$  o  $270^\circ$ .

Le CNN, reti neurali convoluzionali, rappresentano una tipologia specializzata di feed-forward neural network, progettate specificamente per elaborare dati strutturati come le immagini.

Il lavoro svolto esplora due approcci distinti per affrontare il problema della classificazione dell'orientamento. Il primo approccio prevede la progettazione e l'addestramento di una CNN personalizzata, con un processo iterativo di ottimizzazione della struttura e dei parametri per massimizzare le performance del modello. Il secondo approccio adotta una strategia diversa: utilizzando una CNN pre-addestrata come feature extractor, le caratteristiche estratte vengono successivamente classificate mediante Support Vector Machine (SVM).

L'obiettivo principale è confrontare le prestazioni di questi due approcci, valutando vantaggi e limitazioni di ciascuno nel contesto specifico della classificazione dell'orientamento delle immagini.

## 2 Dataset

Il dataset utilizzato per questo progetto è PASCAL VOC2012, un benchmark molto famoso in computer vision, originariamente pensato per task di object detection e recognition. Il dataset contiene immagini raffiguranti 20 categorie di oggetti, tra cui persone, animali e mezzi di trasporto, per un totale di oltre

5000 fotografie.

La scelta di utilizzare PASCAL VOC2012, piuttosto che un dataset specificamente progettato per la classificazione dell'orientamento, è motivata dall'obiettivo di valutare le prestazioni dei modelli in condizioni realistiche e con molta eterogeneità visiva. Questa scelta introduce una maggiore complessità rispetto a dataset specializzati, ma garantisce una valutazione più robusta della capacità di generalizzazione dei modelli su immagini naturali e di diversi contesti.

## 2.1 Selezione e Preprocessing

Data la limitata disponibilità di risorse computazionali, è stato selezionato un sottoinsieme di 600 immagini dal dataset originale, poi suddivise in training, validation e test set secondo una proporzione 60%-20%-20%, ottenendo rispettivamente 360, 120 e 120 immagini.

Per garantire uniformità nella forma delle immagini in input alla rete neurale, è stata implementata una strategia di cropping che estrae parti quadrate da ogni immagine. L'algoritmo divide ogni immagine in crop di dimensione pari al lato minore dell'immagine originale, con sovrapposizione in modo da massimizzare lo spazio usato. Questo processo genera, per ogni foto, 2 crop, raddoppiando la cardinalità degli insiemi di training, validation e di test.

## 2.2 Data Loading e Augmentation

I data loader implementati gestiscono automaticamente le operazioni di preprocessing, tra cui normalizzazione dei valori dei pixel nel range  $[0,1]$  e, nel caso del training set, applicazione di una delle possibili rotazioni ( $0^\circ$ ,  $90^\circ$ ,  $180^\circ$ ,  $270^\circ$ ) ad ogni immagine durante il caricamento. Questo approccio di augmentation permette al modello di vedere ogni crop in tutti i possibili orientamenti durante l'addestramento.

Per il test set, invece, ogni immagine viene ruotata in tutti e 4 i modi possibili, creando così un dataset di valutazione finale di 960 immagini ( $240 \times 4$ ). Questo approccio garantisce un test set completamente bilanciato tra le quattro classi e permette di valutare ottimamente la capacità del modello di riconoscere ogni orientamento.

### 3 Approccio metodologico

Il lavoro si è svolto principalmente con 2 diversi approcci:

- un approccio "stile assignment" svolti durante il corso, quindi con una cnn personalizzata
- il secondo invece, è stato fare feature extractor per poi fare un semplice task di classificazione usando SVM

#### 3.1 CNN da zero

L'idea di base qua, è creare ed addestrare una CNN da zero, cercando di migliorare le performance passo-passo.

Per tutti i modelli è stato usato come optimizer Adam (lr=0.0001)

I risultati saranno descritti nel capitolo 4.

##### 3.1.1 Primo modello

Il primo modello è una semplicissima CNN di partenza, usata come guideline per i successivi miglioramenti, con un numero totale di parametri di 835.950

Table 1: Struttura 'modell1'

| Layer               | Output shape | Params |
|---------------------|--------------|--------|
| conv2d              | (111,111,16) | 448    |
| batch normalization | (111,111,16) | 64     |
| conv2d              | (55,55,32)   | 4640   |
| batch normalization | (55,55,32)   | 128    |
| conv2d              | (27,27,64)   | 18496  |
| batch normalization | (27,27,64)   | 256    |
| conv2d              | (13,13,128)  | 73856  |
| dropout = 0.2       | (13,13,128)  | 0      |
| conv2d              | (6,6,128)    | 147584 |
| dropout=0.5         | (6,6,128)    | 0      |
| flatten             | (4608)       | 0      |
| dense               | (128)        | 589592 |
| dropout=0.5         | (128)        | 0      |
| dense               | (4)          | 516    |

##### 3.1.2 Secondo modello

Il secondo modello introduce tre modifiche principali: riduzione del learning rate da 0.001 a 0.0001 per una convergenza più graduale, dropout meno

aggressivo per favorire l'apprendimento e aumento della dimensione dei filtri convoluzionali. L'architettura risultante conta 1.192.772 parametri, di seguito un riassunto architetturale:

Table 2: Struttura 'model2'

| Layer               | Output shape | Params  |
|---------------------|--------------|---------|
| conv2d              | (112,112,32) | 896     |
| batch normalization | (112,112,32) | 128     |
| conv2d              | (56,56,64)   | 18.496  |
| batch normalization | (56,56,64)   | 256     |
| conv2d              | (28,28,128)  | 73.856  |
| batch normalization | (28,28,128)  | 512     |
| conv2d              | (14,14,128)  | 147.584 |
| dropout = 0.3       | (14,14,128)  | 0       |
| conv2d              | (7,7,128)    | 147.584 |
| dropout=0.4         | (7,7,128)    | 0       |
| flatten             | (6272)       | 0       |
| dense               | (128)        | 802.944 |
| dropout=0.4         | (128)        | 0       |
| dense               | (4)          | 516     |

### 3.1.3 Terzo modello

Dal terzo modello la batch size passa a 32 per stabilizzare il gradiente, le epoche diventano 50 e il dropout adotta una strategia graduale, ovvero crescente con la profondità della rete.

Table 3: Struttura 'model3'

| Layer               | Output shape | Params  |
|---------------------|--------------|---------|
| conv2d              | (112,112,32) | 896     |
| batch normalization | (112,112,32) | 128     |
| conv2d              | (56,56,64)   | 18.496  |
| batch normalization | (56,56,64)   | 256     |
| conv2d              | (28,28,128)  | 73.856  |
| batch normalization | (28,28,128)  | 512     |
| conv2d              | (14,14,128)  | 147.584 |
| dropout = 0.2       | (14,14,128)  | 0       |
| conv2d              | (7,7,128)    | 147.584 |
| dropout=0.4         | (7,7,128)    | 0       |
| flatten             | (6272)       | 0       |
| dense               | (128)        | 802.944 |
| dropout=0.5         | (128)        | 0       |
| dense               | (4)          | 516     |

### 3.1.4 Quarto modello

Nel quarto modello l'ultimo layer utilizza 256 filtri per aumentare la capacità rappresentativa. Lo stride è limitato e si fa uso di MaxPooling2D (2,2) per un downsampling più controllato. Un'ulteriore data augmentation su luminosità e contrasto ( $\pm 20\%$ ) non ha portato benefici ed è stata quindi scartata. Parametri totali: 6.960.068

Table 4: Struttura 'model4'

| Layer               | Output shape | Params    |
|---------------------|--------------|-----------|
| conv2d              | (112,112,32) | 896       |
| batch normalization | (112,112,32) | 128       |
| conv2d              | (56,56,64)   | 18.496    |
| batch normalization | (56,56,64)   | 256       |
| conv2d              | (56,56,128)  | 73.856    |
| batch normalization | (56,56,128)  | 512       |
| max pooling         | (28,28,128)  | 0         |
| conv2d              | (28,28,128)  | 147.584   |
| dropout = 0.2       | (28,28,128)  | 0         |
| max pooling         | (14,14,128)  | 0         |
| conv2d              | (14,14,256)  | 295.168   |
| dropout=0.4         | (14,14,256)  | 0         |
| flatten             | (50176)      | 0         |
| dense               | (128)        | 6.422.656 |
| dropout=0.5         | (128)        | 0         |
| dense               | (4)          | 516       |

### 3.1.5 Quinto modello

Nel quinto modello è stata introdotta regolarizzazione L2 per ridurre l'overfitting. Dopo un primo test con L2=0.0001 senza effetti significativi, il coefficiente è stato aumentato a 0.001. Architettura e altri iperparametri restano invariati.

Table 5: Struttura 'model5'

| Layer               | Output shape | Params    |
|---------------------|--------------|-----------|
| conv2d              | (112,112,32) | 896       |
| batch normalization | (112,112,32) | 128       |
| conv2d              | (56,56,64)   | 18.496    |
| batch normalization | (56,56,64)   | 256       |
| conv2d              | (56,56,128)  | 73.856    |
| batch normalization | (56,56,128)  | 512       |
| max pooling         | (28,28,128)  | 0         |
| conv2d              | (28,28,128)  | 147.584   |
| dropout = 0.2       | (28,28,128)  | 0         |
| max pooling         | (14,14,128)  | 0         |
| conv2d              | (14,14,256)  | 295.168   |
| dropout=0.4         | (14,14,256)  | 0         |
| flatten             | (50176)      | 0         |
| dense               | (128)        | 6.422.656 |
| dropout=0.5         | (128)        | 0         |
| dense               | (4)          | 516       |

## 3.2 Feature Extraction e SVM Classification

Il secondo approccio adottato si basa sull'uso di una CNN pre-addestrata come feature extractor, seguita da un classificatore SVM per la classificazione delle feature estratte.

### 3.2.1 Scelta del Feature Extractor

La selezione della rete pre-addestrata è stata effettuata considerando tre architetture discusse durante il corso:

- **MobileNet**: rete pre-addestrata su ImageNet, caratterizzata da un'architettura leggera e computazionalmente efficiente, particolarmente adatta a contesti con risorse limitate.
- **ResNet**: più potente in termini di capacità rappresentativa, ma con maggiore complessità computazionale.
- **VGG**: scartata a priori per l'elevato numero di parametri e limiti temporali/risorse.

Sulla base di queste considerazioni, è stata selezionata MobileNet.

### 3.2.2 Feature Extraction

Una volta caricata la rete MobileNet pre-addestrata, è stata rimossa la parte di classificazione finale, utilizzando i layer convoluzionali come estrattore di feature. Le rappresentazioni ad alto livello sono state estratte per tutti i tre set: training, validation e test.

### 3.2.3 SVM Classification e Ottimizzazione

Per la fase di classificazione è stata usata una SVM. Gli iperparametri ottimali sono stati identificati mediante Grid Search con cross-validation. La configurazione migliore risultante è:

- $C = 10$
- kernel = RBF
- gamma = scale

Il modello è stato validato tramite k-fold cross-validation sul training set per verificarne la stabilità e la capacità di generalizzazione. Successivamente, il training finale è stato eseguito sul set combinato di training e validation, per massimizzare l'utilizzo dei dati disponibili.

## 4 Risultati e valutazioni

### 4.1 CNN da zero

#### 4.1.1 Primo modello

Il primo modello ha prodotto risultati insufficienti su tutti gli indicatori di performance. Il training è stato interrotto dopo sole 12 delle 30 epoche previste a causa della mancanza di miglioramenti. L'accuracy si è mantenuta su valori molto bassi, mentre la loss è rimasta elevata durante tutto l'addestramento. Le curve di apprendimento mostrano un andamento instabile, suggerendo difficoltà del modello nell'apprendere pattern significativi dai dati.

Questi risultati mostrano che servono modifiche sostanziali all'architettura e agli iperparametri.

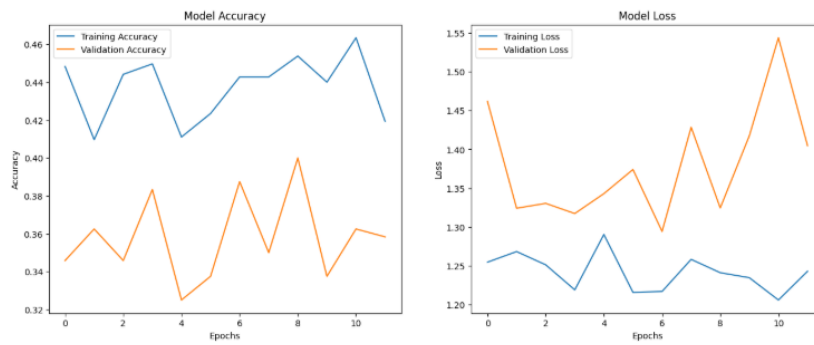


Figure 1: Accuracy e loss di 'model1'

#### 4.1.2 Secondo modello

I risultati mostrano miglioramenti importanti: curve di apprendimento più regolari, accuracy aumentata, con evidente overfitting. La loss di training decresce costantemente, mentre quella di validation si stabilizza dopo poche epoche, indicando una limitata generalizzazione.

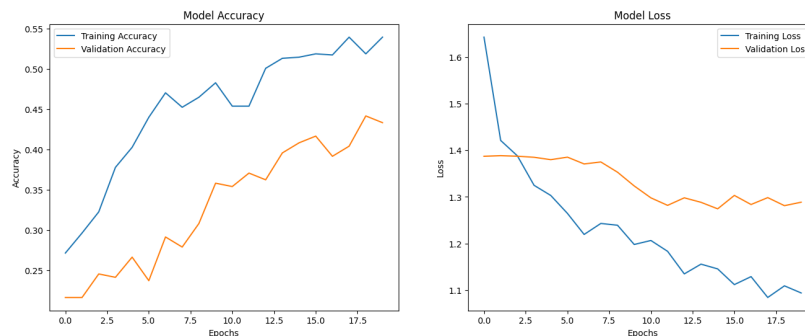


Figure 2: Accuracy e loss di 'model2'

#### 4.1.3 Terzo modello

I risultati mostrano qualche miglioramento quasi impercettibile in accuracy e loss, mentre persiste un forte overfitting.

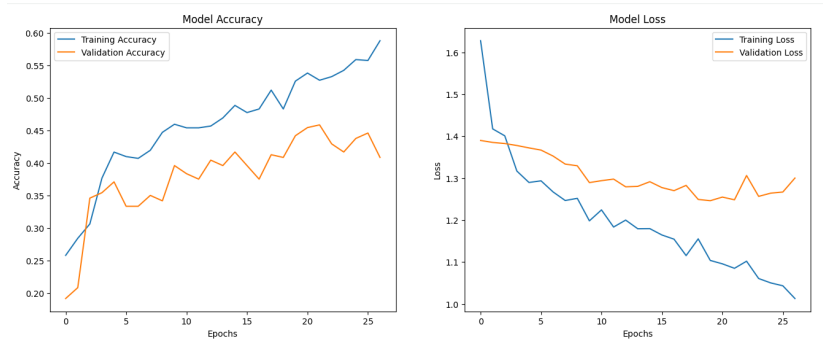


Figure 3: Accuracy e loss di 'model3'

#### 4.1.4 Quarto modello

I risultati seguono il trend precedente: miglioramenti in accuracy e loss, ma overfitting persistente. Le curve di apprendimento mostrano maggiore linearità rispetto ai primi modelli, indicando stabilità nell'ottimizzazione.

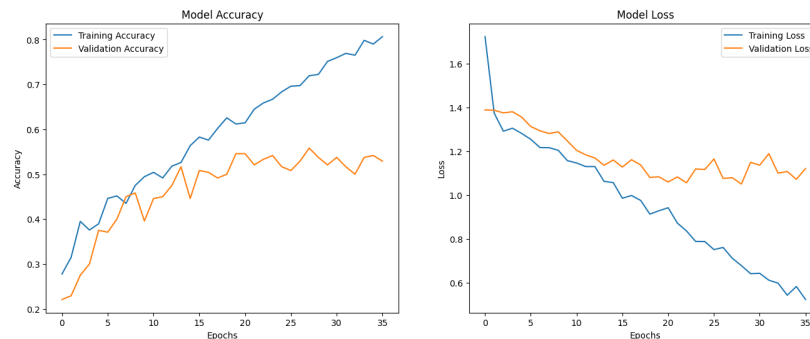


Figure 4: Accuracy e loss di 'model4'

#### 4.1.5 Quinto modello

I risultati sono i migliori ottenuti: un leggero calo di performance è compensato da una riduzione importante di overfitting. L'accuracy si avvicina al 60% ed è un risultato accettabile.

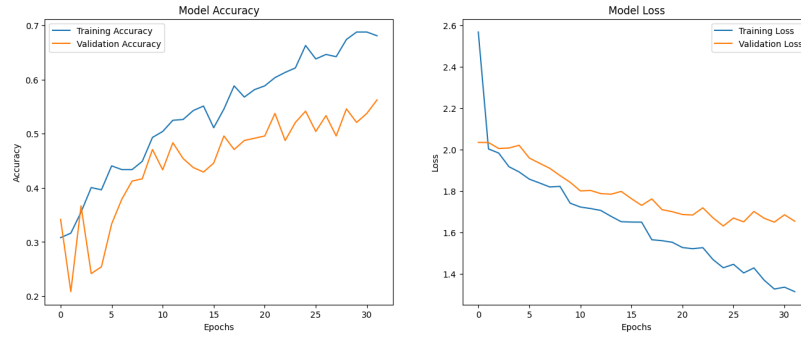


Figure 5: Accuracy e loss di 'model5'

#### 4.1.6 Valutazione confusion matrix

Le confusion matrix dei cinque modelli sono riportate di seguito per mostrare l'evoluzione delle performance con l'evoluzione dei modelli

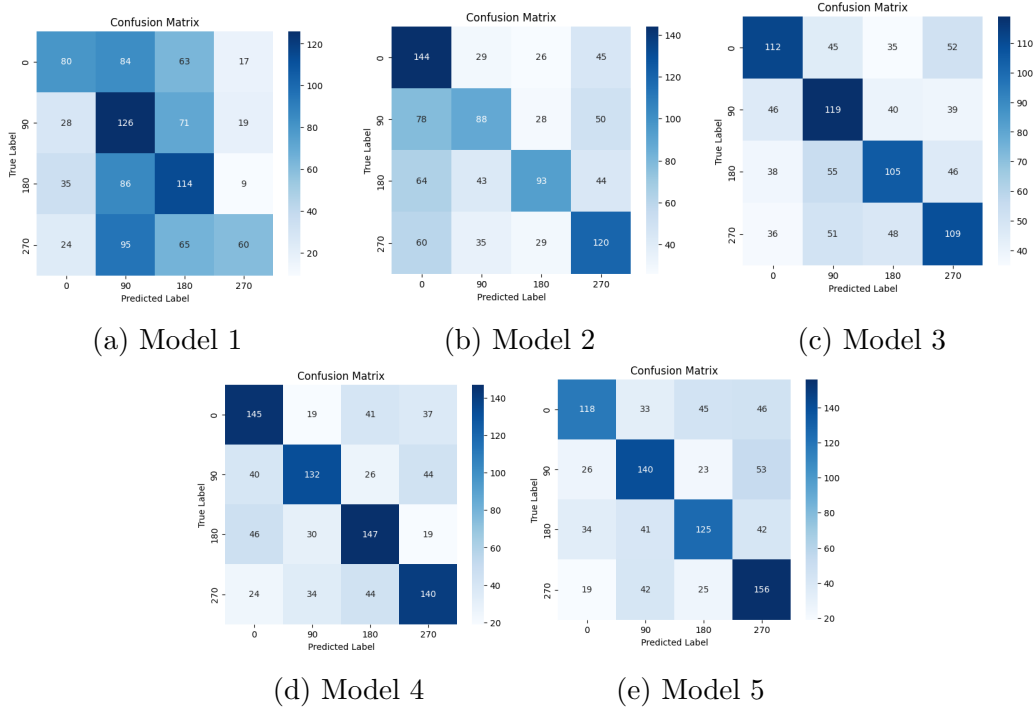


Figure 6: Confusion matrices dei modelli

## 4.2 Feature Extractor + SVM

Di seguito i risultati della SVM usando per il training anche i dati di validation:

Table 6: Metriche SVM

| Classe | Precision | Recall | F1   |
|--------|-----------|--------|------|
| 0      | 0.83      | 0.86   | 0.84 |
| 1      | 0.46      | 0.46   | 0.46 |
| 2      | 0.79      | 0.83   | 0.81 |
| 3      | 0.45      | 0.42   | 0.43 |

Accuracy : 64%

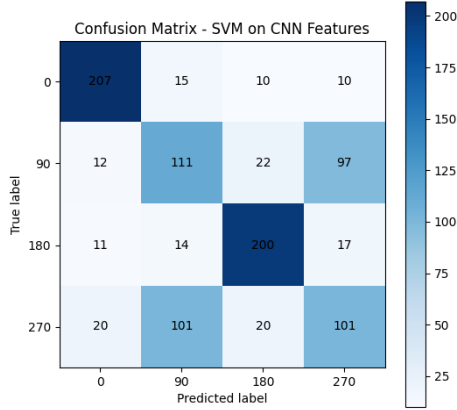


Figure 7: Confusion matrix SVM

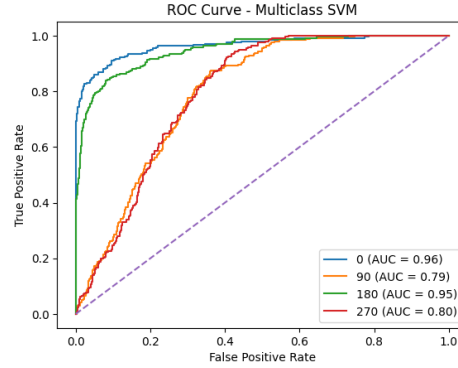


Figure 8: Curve ROC delle 4 classi

I grafici indicano prestazioni migliori della CNN personalizzata, ma con una forte asimmetria tra le classi:  $0^\circ$  e  $180^\circ$  risultano molto più accurate di  $90^\circ$  e  $270^\circ$ . Ciò dipende dal pre-addestramento di MobileNet su immagini con orientamento standard, che rende più facili le classificazioni  $0^\circ$ – $180^\circ$  e più difficili quelle ortogonali.

## 5 Discussioni e conclusioni

### 5.1 Interpretazione dei Risultati

L'approccio CNN personalizzato ha mostrato come le modifiche progressive su learning rate, dropout, batch size e regolarizzazione L2 e la scelta accurata

degli iperparametri siano cruciali per il successo del training.

L'approccio con MobileNet e SVM ha dimostrato performance superiori, confermando l'efficacia delle feature pre-addestrate. Questo risultato supporta l'ipotesi che le rappresentazioni apprese su dataset di grandi dimensioni come ImageNet possiedano caratteristiche generali sufficientemente robuste da essere trasferibili a compiti di classificazione diversi.

L'utilizzo di PASCAL VOC2012, dataset non specificamente progettato per la classificazione dell'orientamento, ha introdotto complessità aggiuntiva ma ha garantito una valutazione più realistica della capacità di generalizzazione dei modelli.

## 5.2 Miglioramenti Futuri

- **Data augmentation** : implementare ulteriori strategie di augmentation, ad esempio attraverso rotazioni multiple durante il training, inserire immagini con maggiore/minore luminosità,...
- **Analisi approfondita degli errori**: investigare i casi di immagini 'mislabelled' per identificare pattern specifici (es. immagini con contenuto simmetrico, scene complesse) che causano maggiore confusione.
- **Ottimizzazione delle risorse**: testare l'approccio su dataset più ampi e con architetture più profonde, cercando di superare i vincoli computazionali

## 5.3 Conclusioni

Il confronto tra i due approcci ha evidenziato che, nel contesto di risorse limitate e dataset di dimensioni moderate, l'apprendimento con feature extraction rappresenta una strategia più efficiente rispetto al training end-to-end di una CNN personalizzata. Tuttavia, il training iterativo della CNN ha fornito insights importanti sull'apprendimento e su problemi comuni (overfitting, scelta degli iperparametri..) che caratterizzano lo sviluppo di modelli deep learning.